

Reg No: 240968040

Name: Devarjya

Q1) Write a parallel program using OpenMP to implement the Selection sort algorithm. Compute the efficiency and plot the speed up for varying input size and thread number

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void selection_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        int min_val = arr[i];

        #pragma omp parallel for reduction(min:min_val)
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < min_val) {
                min_val = arr[j];
            }
        }

        for (int j = i + 1; j < n; j++) {
            if (arr[j] == min_val) {
                min_idx = j;
                break;
            }
        }

        if (min_idx != i) {
            int temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}

int main() {
    // Varying input sizes
    int input_sizes[] = {5000, 10000, 20000};
    int num_sizes = sizeof(input_sizes) / sizeof(input_sizes[0]);

    int threads = omp_get_max_threads();
    printf("Reg No: 240968040, Name: Devarjya\n");
    printf("Input Size\tSeq Time (s)\tPar Time (s)\tSpeedup\tEfficiency\n");
    printf("-----\n");

    for (int s = 0; s < num_sizes; s++) {
        int n = input_sizes[s];
        int *arr = (int *)malloc(n * sizeof(int));
```

```

// Fill array with random data
for (int i = 0; i < n; i++) arr[i] = rand() % 10000;

// 1. Measure Sequential Time (1 thread)
omp_set_num_threads(1);
double start_seq = omp_get_wtime();
selection_sort(arr, n);
double seq_time = omp_get_wtime() - start_seq;

// Refill array with fresh random data for parallel sort
for (int i = 0; i < n; i++) arr[i] = rand() % 10000;

// 2. Measure Parallel Time (using external OMP_NUM_THREADS)
omp_set_num_threads(threads);
double start_par = omp_get_wtime();
selection_sort(arr, n);
double par_time = omp_get_wtime() - start_par;

if (par_time == 0.0) par_time = 0.00001; // Avoid division by zero

// 3. Compute Metrics
double speedup = seq_time / par_time;
double efficiency = speedup / threads;

printf("%d\t\t%.4f\t\t%.4f\t\t%.2f\t\t%.2f\n",
       n, seq_time, par_time, speedup, efficiency);

free(arr);
}

return 0;
}

```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=1
pcap1@PC-36:~/Documents/240968040/week 3$ ./q1
Reg No: 240968040, Name: Devarjya
```

Input Size	Seq Time (s)	Par Time (s)	Speedup	Efficiency
5000	0.0337	0.0152	2.22	2.22
10000	0.0574	0.0576	1.00	1.00
20000	0.2218	0.2228	1.00	1.00

```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=2
pcap1@PC-36:~/Documents/240968040/week 3$ ./q1
Reg No: 240968040, Name: Devarjya
```

Input Size	Seq Time (s)	Par Time (s)	Speedup	Efficiency
5000	0.0239	0.0138	1.73	0.86
10000	0.0580	0.0466	1.24	0.62
20000	0.2281	0.1587	1.44	0.72

```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=4
pcap1@PC-36:~/Documents/240968040/week 3$ ./q1
Reg No: 240968040, Name: Devarjya
```

Input Size	Seq Time (s)	Par Time (s)	Speedup	Efficiency
5000	0.0191	0.0126	1.52	0.38
10000	0.0478	0.0363	1.32	0.33
20000	0.1837	0.1204	1.53	0.38

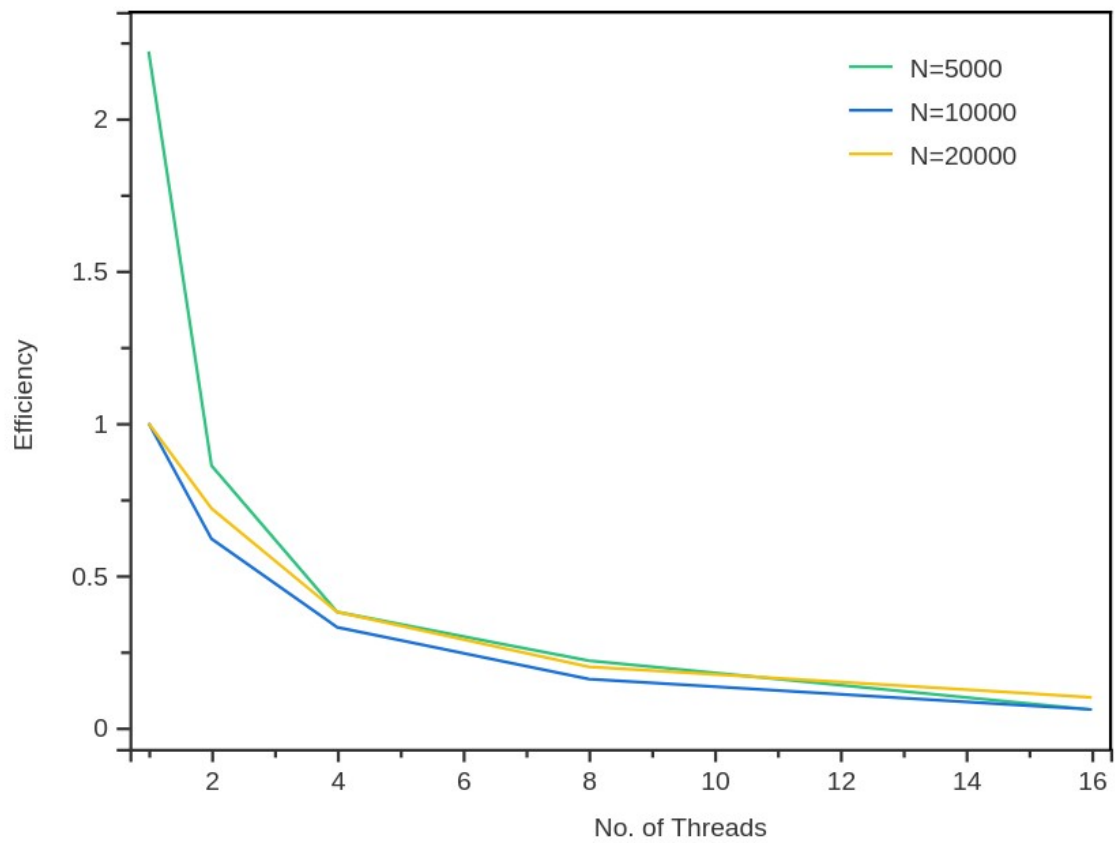
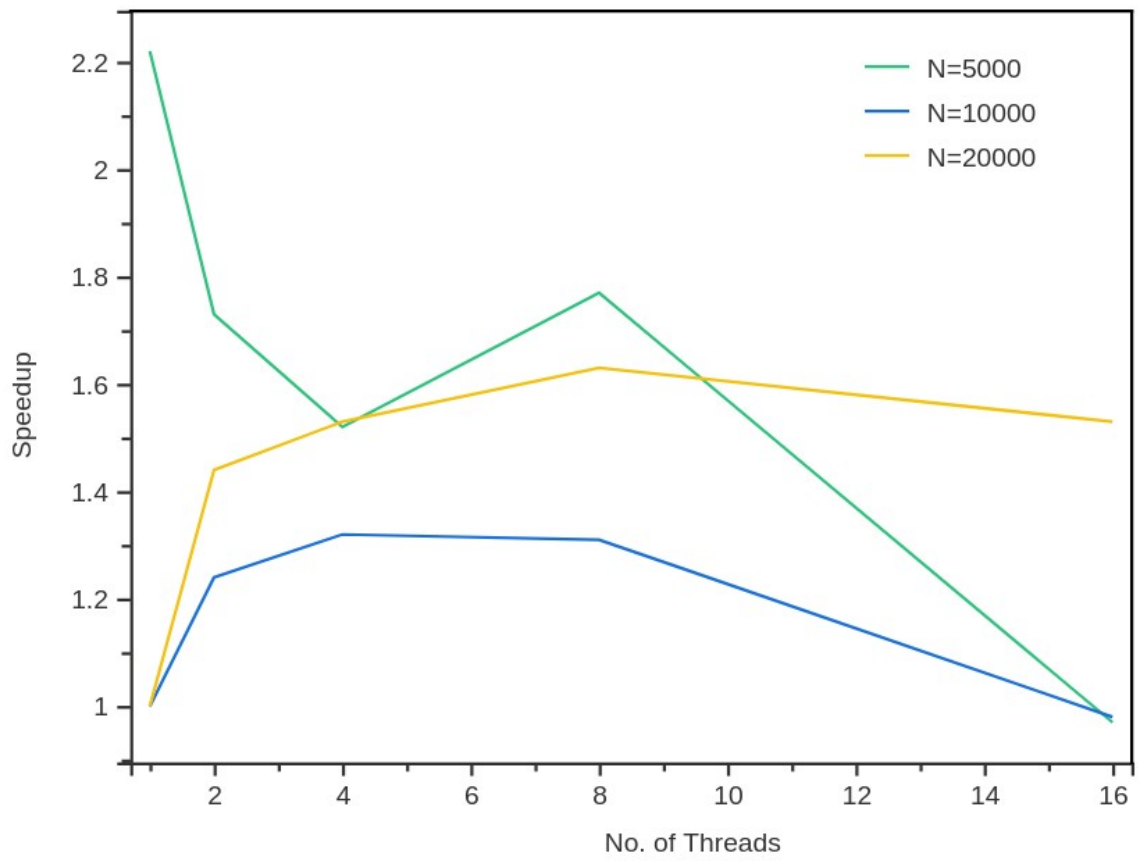
```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=8
pcap1@PC-36:~/Documents/240968040/week 3$ ./q1
Reg No: 240968040, Name: Devarjya
```

Input Size	Seq Time (s)	Par Time (s)	Speedup	Efficiency
5000	0.0276	0.0156	1.77	0.22
10000	0.0480	0.0366	1.31	0.16
20000	0.1843	0.1134	1.63	0.20

```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=16
pcap1@PC-36:~/Documents/240968040/week 3$ ./q1
Reg No: 240968040, Name: Devarjya
```

Input Size	Seq Time (s)	Par Time (s)	Speedup	Efficiency
5000	0.0194	0.0200	0.97	0.06
10000	0.0594	0.0604	0.98	0.06
20000	0.1882	0.1228	1.53	0.10

Plot:
(N is input size)



Q2) Write a parallel program using OpenMP to implement sequential search algorithm. Compute the efficiency and plot the speed up for varying input size and thread number.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int parallel_search(int arr[], int n, int target) {
    int found_index = -1;

    #pragma omp parallel for reduction(max:found_index)
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            // Keep track of the index if found (max reduction handles finding any valid index)
            if (i > found_index) {
                found_index = i;
            }
        }
    }
    return found_index;
}

// Sequential Search for baseline timing
int sequential_search(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}

int main() {
    // Varying input sizes
    int input_sizes[] = {500000, 1000000, 2000000};
    int num_sizes = sizeof(input_sizes) / sizeof(input_sizes[0]);

    int threads = omp_get_max_threads();
    printf("Reg No: 240968040, Name: Devarjya\n");
    printf("Input Size\tSeq Time (s)\tPar Time (s)\tSpeedup\tEfficiency\n");
    printf("-----\n");

    for (int s = 0; s < num_sizes; s++) {
        int n = input_sizes[s];
        int *arr = (int *)malloc(n * sizeof(int));

        // Fill array with random data and choose a target that exists (e.g., last element)
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        int target = n - 1; // Worst-case scenario for search
```

```

// 1. Measure Sequential Time (1 thread)
omp_set_num_threads(1);
double start_seq = omp_get_wtime();
sequential_search(arr, n, target);
double seq_time = omp_get_wtime() - start_seq;
if (seq_time == 0.0) seq_time = 0.00001;

// 2. Measure Parallel Time (using external OMP_NUM_THREADS)
omp_set_num_threads(threads);
double start_par = omp_get_wtime();
parallel_search(arr, n, target);
double par_time = omp_get_wtime() - start_par;
if (par_time == 0.0) par_time = 0.00001;

// 3. Compute Metrics
double speedup = seq_time / par_time;
double efficiency = speedup / threads;

printf("%d\t\t%.4f\t\t%.4f\t\t%.2f\t\t%.2f\n",
       n, seq_time, par_time, speedup, efficiency);

free(arr);
}

return 0;
}

```

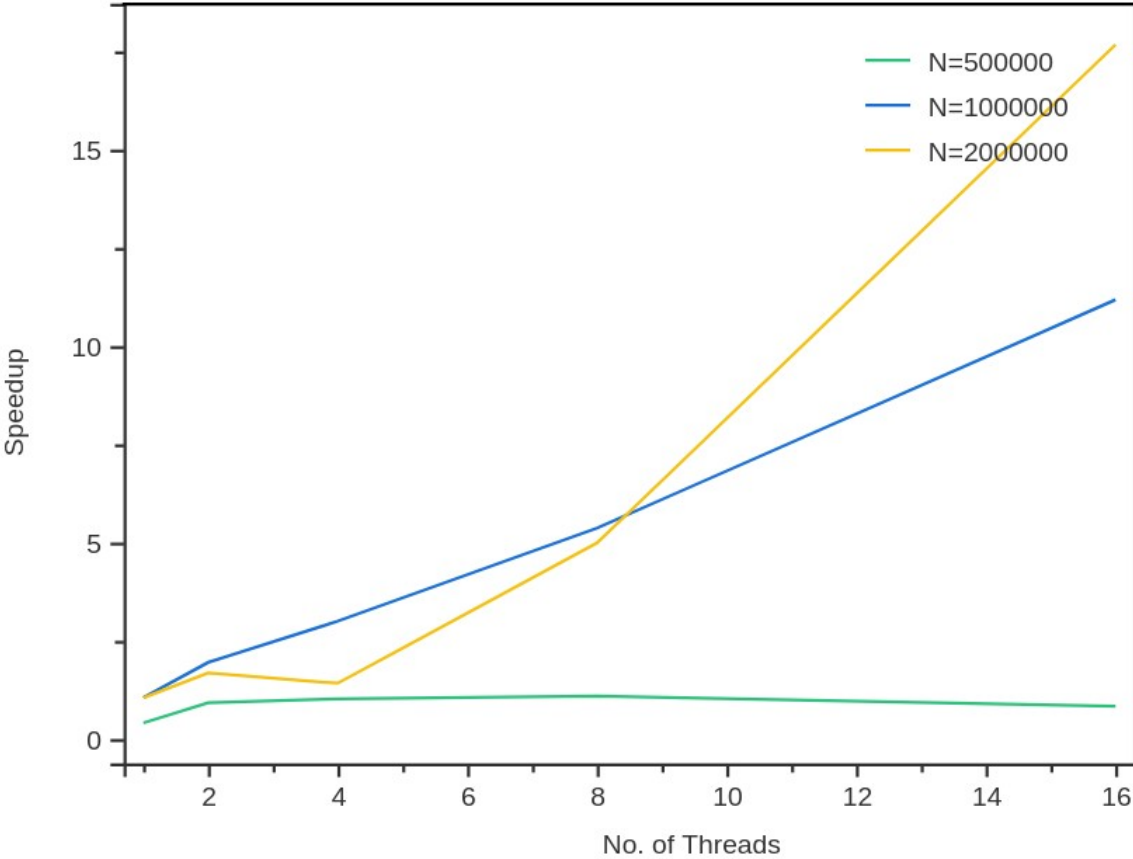
Output:

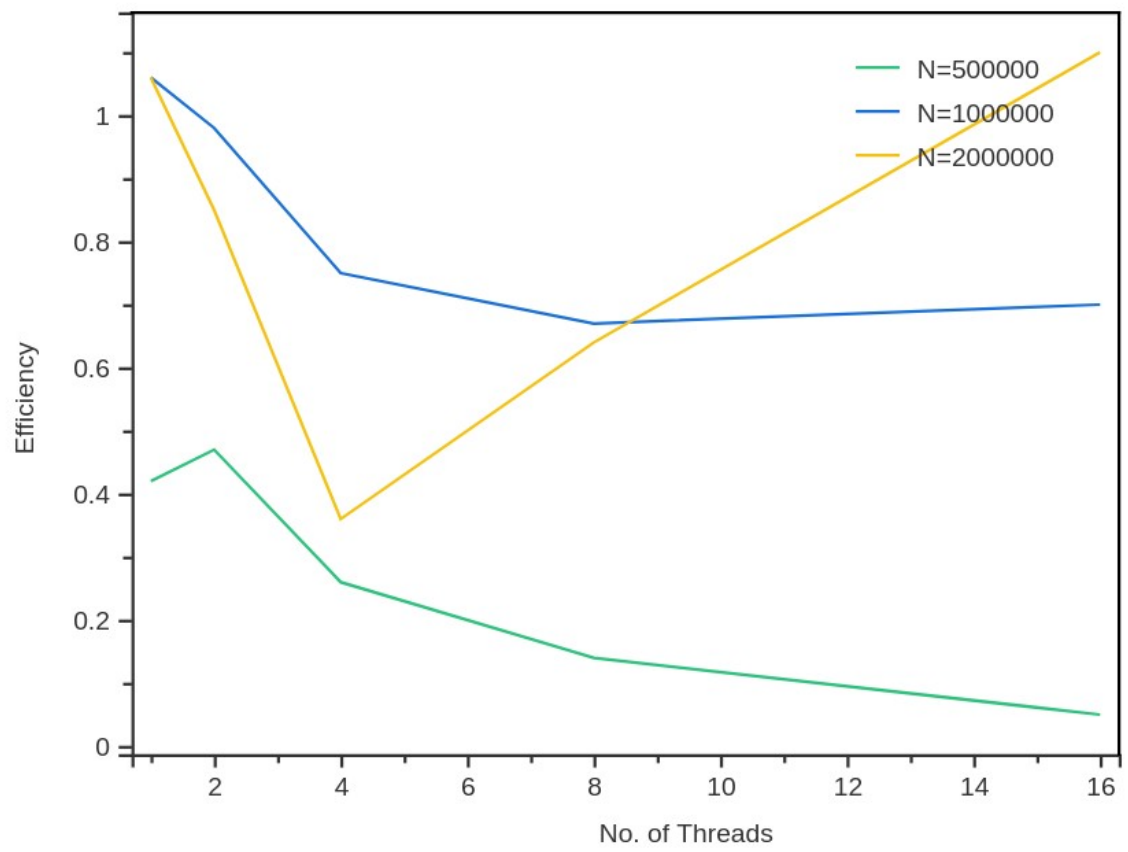
```

pcap1@PC-36:~/Documents/240968040/week 3$ gcc -fopenmp q2.c -o q2
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=1
pcap1@PC-36:~/Documents/240968040/week 3$ ./q2
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)      Par Time (s)      Speedup      Efficiency
-----
500000          0.0009            0.0022            0.42         0.42
1000000         0.0006            0.0006            1.06         1.06
2000000         0.0012            0.0012            1.06         1.06
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=2
pcap1@PC-36:~/Documents/240968040/week 3$ ./q2
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)      Par Time (s)      Speedup      Efficiency
-----
500000          0.0010            0.0011            0.93         0.47
1000000         0.0006            0.0003            1.96         0.98
2000000         0.0012            0.0007            1.69         0.85
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=4
pcap1@PC-36:~/Documents/240968040/week 3$ ./q2
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)      Par Time (s)      Speedup      Efficiency
-----
500000          0.0010            0.0010            1.03         0.26
1000000         0.0006            0.0002            3.01         0.75
2000000         0.0013            0.0009            1.43         0.36
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=8
pcap1@PC-36:~/Documents/240968040/week 3$ ./q2
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)      Par Time (s)      Speedup      Efficiency
-----
500000          0.0009            0.0008            1.10         0.14
1000000         0.0008            0.0002            5.38         0.67
2000000         0.0016            0.0003            5.11         0.64
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=16
pcap1@PC-36:~/Documents/240968040/week 3$ ./q2
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)      Par Time (s)      Speedup      Efficiency
-----
500000          0.0009            0.0011            0.84         0.05
1000000         0.0011            0.0001            11.19        0.70
2000000         0.0022            0.0001            17.68        1.10
pcap1@PC-36:~/Documents/240968040/week 3$

```

Plot:
(N is input size)





Q3) Write a parallel program using OpenMP to perform vector addition, subtraction, multiplication. Demonstrate task level parallelism. Analyze the speedup and efficiency of the parallelized code

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
void vector_operations(int *a, int *b, int *add_res, int *sub_res, int *mul_res, int n) {
```

```
    #pragma omp parallel
    {
        #pragma omp single
        {
            #pragma omp task
            {
                for (int i = 0; i < n; i++) {
                    add_res[i] = a[i] + b[i];
                }
            }

            #pragma omp task
            {
                for (int i = 0; i < n; i++) {
                    sub_res[i] = a[i] - b[i];
                }
            }

            #pragma omp task
            {
                for (int i = 0; i < n; i++) {
                    mul_res[i] = a[i] * b[i];
                }
            }
        }
    }
}
```

```
void sequential_vector_operations(int *a, int *b, int *add_res, int *sub_res, int *mul_res, int n) {
    for (int i = 0; i < n; i++) {
        add_res[i] = a[i] + b[i];
    }
    for (int i = 0; i < n; i++) {
        sub_res[i] = a[i] - b[i];
    }
    for (int i = 0; i < n; i++) {
        mul_res[i] = a[i] * b[i];
    }
}
```

```

int main() {
    int input_sizes[] = {500000, 1000000, 2000000};
    int num_sizes = sizeof(input_sizes) / sizeof(input_sizes[0]);

    int threads = omp_get_max_threads();
    printf("Reg No: 240968040, Name: Devarjya\n");
    printf("Input Size\tSeq Time (s)\tPar Time (s)\tSpeedup\tEfficiency\n");
    printf("-----\n");

    for (int s = 0; s < num_sizes; s++) {
        int n = input_sizes[s];

        int *a = (int *)malloc(n * sizeof(int));
        int *b = (int *)malloc(n * sizeof(int));
        int *add_seq = (int *)malloc(n * sizeof(int));
        int *sub_seq = (int *)malloc(n * sizeof(int));
        int *mul_seq = (int *)malloc(n * sizeof(int));
        int *add_par = (int *)malloc(n * sizeof(int));
        int *sub_par = (int *)malloc(n * sizeof(int));
        int *mul_par = (int *)malloc(n * sizeof(int));

        for (int i = 0; i < n; i++) {
            a[i] = rand() % 100;
            b[i] = rand() % 100;
        }

        // 1. Measure Sequential Time (1 thread)
        omp_set_num_threads(1);
        double start_seq = omp_get_wtime();
        sequential_vector_operations(a, b, add_seq, sub_seq, mul_seq, n);
        double seq_time = omp_get_wtime() - start_seq;
        if (seq_time == 0.0) seq_time = 0.00001;

        // 2. Measure Parallel Time (using external OMP_NUM_THREADS with task parallelism)
        omp_set_num_threads(threads);
        double start_par = omp_get_wtime();
        vector_operations(a, b, add_par, sub_par, mul_par, n);
        double par_time = omp_get_wtime() - start_par;
        if (par_time == 0.0) par_time = 0.00001;

        // 3. Compute Metrics
        double speedup = seq_time / par_time;
        double efficiency = speedup / threads;

        printf("%d\t\t%.4f\t\t%.4f\t\t%.2f\t\t%.2f\n",
            n, seq_time, par_time, speedup, efficiency);

        free(a); free(b);
        free(add_seq); free(sub_seq); free(mul_seq);
        free(add_par); free(sub_par); free(mul_par);
    }
}

```

```

    return 0;
}

```

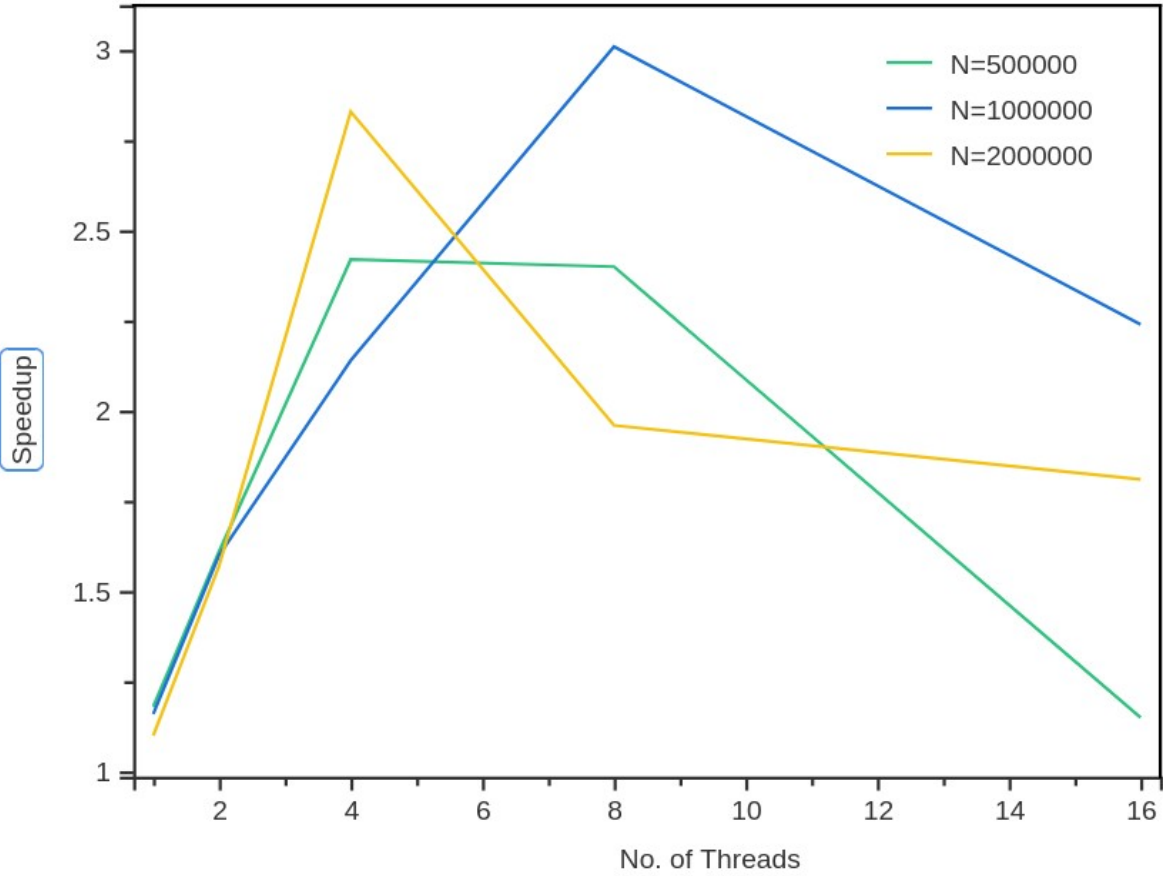
output:

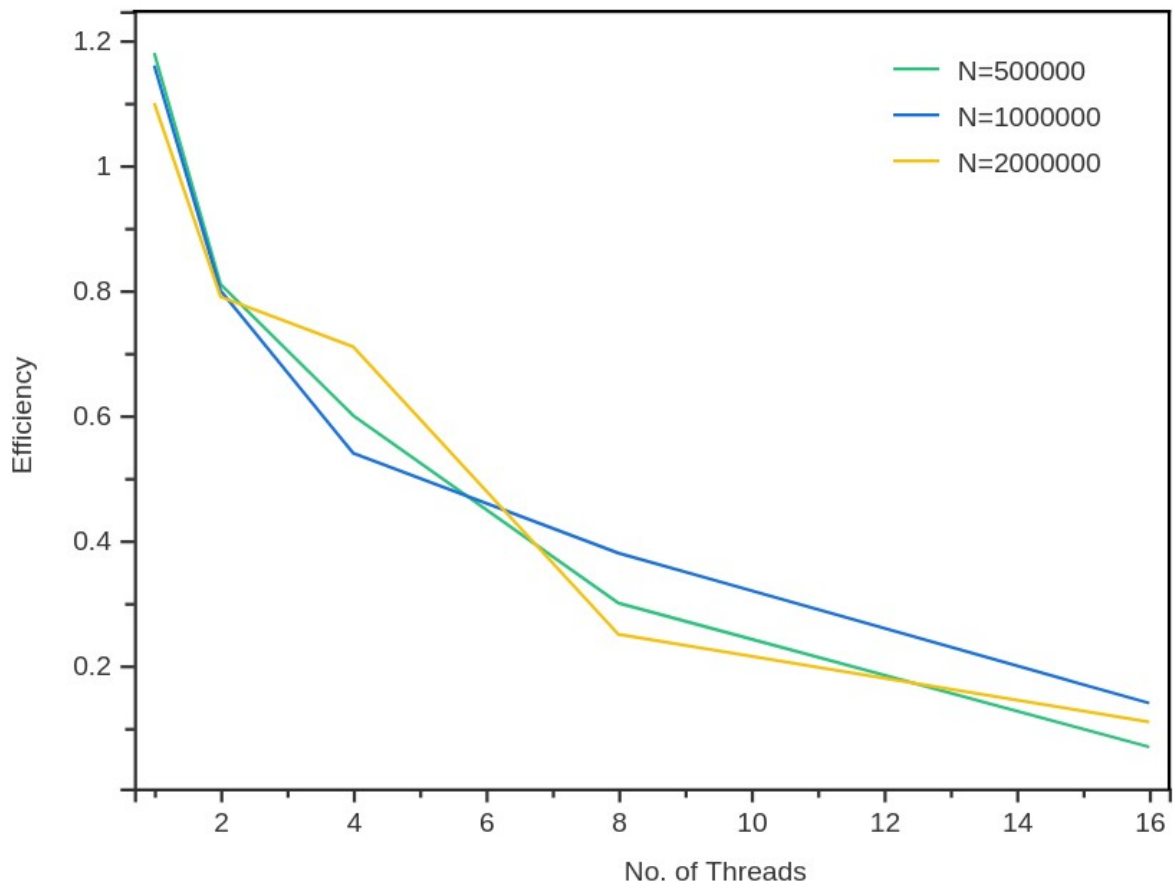
```

pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=1
pcap1@PC-36:~/Documents/240968040/week 3$ ./q3
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)    Par Time (s)    Speedup          Efficiency
-----
500000          0.0039          0.0033          1.18             1.18
1000000         0.0063          0.0054          1.16             1.16
2000000         0.0107          0.0097          1.10             1.10
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=2
pcap1@PC-36:~/Documents/240968040/week 3$ ./q3
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)    Par Time (s)    Speedup          Efficiency
-----
500000          0.0039          0.0024          1.61             0.81
1000000         0.0061          0.0038          1.60             0.80
2000000         0.0111          0.0071          1.57             0.79
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=4
pcap1@PC-36:~/Documents/240968040/week 3$ ./q3
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)    Par Time (s)    Speedup          Efficiency
-----
500000          0.0039          0.0016          2.42             0.60
1000000         0.0062          0.0029          2.14             0.54
2000000         0.0109          0.0038          2.83             0.71
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=8
pcap1@PC-36:~/Documents/240968040/week 3$ ./q3
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)    Par Time (s)    Speedup          Efficiency
-----
500000          0.0038          0.0016          2.40             0.30
1000000         0.0069          0.0023          3.01             0.38
2000000         0.0109          0.0055          1.96             0.25
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=16
pcap1@PC-36:~/Documents/240968040/week 3$ ./q3
Reg No: 240968040, Name: Devarjya
Input Size      Seq Time (s)    Par Time (s)    Speedup          Efficiency
-----
500000          0.0043          0.0037          1.15             0.07
1000000         0.0059          0.0026          2.24             0.14
2000000         0.0102          0.0056          1.81             0.11

```

Plot:
(N is input size)





Q4) Write a parallel program using OpenMP to find sum of N numbers using the following constructs/clauses.

- **Critical section**
- **Atomic**
- **Reduction**
- **Master**
- **Locks**

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

// 1. Using Critical Section

```
long long sum_critical(int *arr, int n) {
    long long total_sum = 0;
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        #pragma omp critical
        {
            total_sum += arr[i];
        }
    }
    return total_sum;
}
```

```
}
```

```
// 2. Using Atomic
```

```
long long sum_atomic(int *arr, int n) {  
    long long total_sum = 0;  
    #pragma omp parallel for  
    for (int i = 0; i < n; i++) {  
        #pragma omp atomic  
        total_sum += arr[i];  
    }  
    return total_sum;  
}
```

```
// 3. Using Reduction
```

```
long long sum_reduction(int *arr, int n) {  
    long long total_sum = 0;  
    #pragma omp parallel for reduction(+:total_sum)  
    for (int i = 0; i < n; i++) {  
        total_sum += arr[i];  
    }  
    return total_sum;  
}
```

```
// 4. Using Master
```

```
long long sum_master(int *arr, int n) {  
    long long total_sum = 0;  
    #pragma omp parallel  
    {  
        long long local_sum = 0;  
        #pragma omp for  
        for (int i = 0; i < n; i++) {  
            local_sum += arr[i];  
        }  
  
        #pragma omp master  
        {  
  
        }  
  
        #pragma omp atomic  
        total_sum += local_sum;  
    }  
    return total_sum;  
}
```

```
// 5. Using Locks
```

```
long long sum_locks(int *arr, int n) {  
    long long total_sum = 0;  
    omp_lock_t lock;  
    omp_init_lock(&lock);  
  
    #pragma omp parallel for
```

```

    for (int i = 0; i < n; i++) {
        omp_set_lock(&lock);
        total_sum += arr[i];
        omp_unset_lock(&lock);
    }

    omp_destroy_lock(&lock);
    return total_sum;
}

int main() {
    int n = 1000000;
    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = 1; // Sum should equal n
    }

    int threads = omp_get_max_threads();
    printf("Reg No: 240968040, Name: Devarjya\n");
    printf("Input Size: %d, Threads: %d\n\n", n, threads);

    double start, end;

    // Test Critical
    start = omp_get_wtime();
    long long res_crit = sum_critical(arr, n);
    end = omp_get_wtime();
    printf("1. Critical Section : Sum = %lld | Time = %.4f s\n", res_crit, end - start);

    // Test Atomic
    start = omp_get_wtime();
    long long res_atom = sum_atomic(arr, n);
    end = omp_get_wtime();
    printf("2. Atomic          : Sum = %lld | Time = %.4f s\n", res_atom, end - start);

    // Test Reduction
    start = omp_get_wtime();
    long long res_red = sum_reduction(arr, n);
    end = omp_get_wtime();
    printf("3. Reduction          : Sum = %lld | Time = %.4f s\n", res_red, end - start);

    // Test Master
    start = omp_get_wtime();
    long long res_mast = sum_master(arr, n);
    end = omp_get_wtime();
    printf("4. Master              : Sum = %lld | Time = %.4f s\n", res_mast, end - start);

    // Test Locks
    start = omp_get_wtime();
    long long res_lock = sum_locks(arr, n);
    end = omp_get_wtime();
    printf("5. Locks                : Sum = %lld | Time = %.4f s\n", res_lock, end - start);

```



```

    free(arr);
    return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 3$ ./q4
Reg No: 240968040, Name: Devarjya
Input Size: 1000000, Threads: 2

1. Critical Section : Sum = 1000000 | Time = 0.0194 s
2. Atomic           : Sum = 1000000 | Time = 0.0186 s
3. Reduction        : Sum = 1000000 | Time = 0.0010 s
4. Master           : Sum = 1000000 | Time = 0.0010 s
5. Locks            : Sum = 1000000 | Time = 0.0409 s
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=4
pcap1@PC-36:~/Documents/240968040/week 3$ ./q4
Reg No: 240968040, Name: Devarjya
Input Size: 1000000, Threads: 4

1. Critical Section : Sum = 1000000 | Time = 0.0582 s
2. Atomic           : Sum = 1000000 | Time = 0.0169 s
3. Reduction        : Sum = 1000000 | Time = 0.0006 s
4. Master           : Sum = 1000000 | Time = 0.0005 s
5. Locks            : Sum = 1000000 | Time = 0.0826 s
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=8
pcap1@PC-36:~/Documents/240968040/week 3$ ./q4
Reg No: 240968040, Name: Devarjya
Input Size: 1000000, Threads: 8

1. Critical Section : Sum = 1000000 | Time = 0.0951 s
2. Atomic           : Sum = 1000000 | Time = 0.0185 s
3. Reduction        : Sum = 1000000 | Time = 0.0003 s
4. Master           : Sum = 1000000 | Time = 0.0003 s
5. Locks            : Sum = 1000000 | Time = 0.1072 s

```

q

Q5) Write an OpenMP program to find the Summation of integers from a given interval. Analyze the performance of various iteration scheduling strategies

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

```

```

long long compute_sum_scheduled(long long start, long long end, int schedule_type, int
chunk_size) {
    long long total_sum = 0;
    long long n = end - start + 1;

```

```

if (schedule_type == 1) {
    // Static Scheduling
    #pragma omp parallel for reduction(+:total_sum) schedule(static, chunk_size)
    for (long long i = 0; i < n; i++) {
        total_sum += (start + i);
    }
} else if (schedule_type == 2) {
    // Dynamic Scheduling
    #pragma omp parallel for reduction(+:total_sum) schedule(dynamic, chunk_size)
    for (long long i = 0; i < n; i++) {
        total_sum += (start + i);
    }
} else if (schedule_type == 3) {
    // Guided Scheduling
    #pragma omp parallel for reduction(+:total_sum) schedule(guided, chunk_size)
    for (long long i = 0; i < n; i++) {
        total_sum += (start + i);
    }
} else if (schedule_type == 4) {
    // Runtime Scheduling
    #pragma omp parallel for reduction(+:total_sum) schedule(runtime)
    for (long long i = 0; i < n; i++) {
        total_sum += (start + i);
    }
}

return total_sum;
}

int main() {
    long long interval_start = 1;
    long long interval_end = 1000000;
    int chunk_size = 100;
    int threads = omp_get_max_threads();

    printf("Reg No: 240968040, Name: Devarjya\n");
    printf("Interval: [%lld, %lld] | Threads: %d | Chunk Size: %d\n", interval_start, interval_end,
threads, chunk_size);
    printf("-----\n");
    printf("Scheduling Strategy\tTime (s)\tSum Result\n");
    printf("-----\n");

    // 1. Static Schedule
    double start_time = omp_get_wtime();
    long long sum1 = compute_sum_scheduled(interval_start, interval_end, 1, chunk_size);
    double time_static = omp_get_wtime() - start_time;
    printf("Static (chunk %d)\t%.4f s\t%lld\n", chunk_size, time_static, sum1);

    // 2. Dynamic Schedule
    start_time = omp_get_wtime();
    long long sum2 = compute_sum_scheduled(interval_start, interval_end, 2, chunk_size);

```

```
double time_dynamic = omp_get_wtime() - start_time;
printf("Dynamic (chunk %d)\t%.4f s\t%lld\n", chunk_size, time_dynamic, sum2);

// 3. Guided Schedule
start_time = omp_get_wtime();
long long sum3 = compute_sum_scheduled(interval_start, interval_end, 3, chunk_size);
double time_guided = omp_get_wtime() - start_time;
printf("Guided (chunk %d)\t%.4f s\t%lld\n", chunk_size, time_guided, sum3);

// 4. Runtime Schedule
start_time = omp_get_wtime();
long long sum4 = compute_sum_scheduled(interval_start, interval_end, 4, 0);
double time_runtime = omp_get_wtime() - start_time;
printf("Runtime (OMP_SCHEDULE)\t%.4f s\t%lld\n", time_runtime, sum4);

return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=2
pcap1@PC-36:~/Documents/240968040/week 3$ ./q5
Reg No: 240968040, Name: Devarjya
Interval: [1, 1000000] | Threads: 2 | Chunk Size: 100
```

Scheduling Strategy	Time (s)	Sum Result
Static (chunk 100)	0.0029 s	500000500000
Dynamic (chunk 100)	0.0027 s	500000500000
Guided (chunk 100)	0.0014 s	500000500000
Runtime (OMP_SCHEDULE)	0.0159 s	500000500000

```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=4
pcap1@PC-36:~/Documents/240968040/week 3$ ./q5
Reg No: 240968040, Name: Devarjya
Interval: [1, 1000000] | Threads: 4 | Chunk Size: 100
```

Scheduling Strategy	Time (s)	Sum Result
Static (chunk 100)	0.0020 s	500000500000
Dynamic (chunk 100)	0.0019 s	500000500000
Guided (chunk 100)	0.0016 s	500000500000
Runtime (OMP_SCHEDULE)	0.0128 s	500000500000

```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=8
pcap1@PC-36:~/Documents/240968040/week 3$ ./q5
Reg No: 240968040, Name: Devarjya
Interval: [1, 1000000] | Threads: 8 | Chunk Size: 100
```

Scheduling Strategy	Time (s)	Sum Result
Static (chunk 100)	0.0016 s	500000500000
Dynamic (chunk 100)	0.0018 s	500000500000
Guided (chunk 100)	0.0009 s	500000500000
Runtime (OMP_SCHEDULE)	0.0166 s	500000500000

Q6) Write a parallel program using OpenMP to compute π using random shooting.
Hint: Shoot randomly into a square $[0, 1] \times [0, 1]$ and count how many shots hit inside the unit circle and how many do not. Then calculate the ratio of number of points lied inside the circle and total number of generated points. We know that, area of the circle is πr^2 and area of the square is $4r^2$. Now, for a very large number of generated points.

$$\text{Pi} = 4 * (\text{no of points generated inside the circle}) / (\text{number of points generated inside the square})$$

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
int main() {
    long long total_points = 10000000; // 10 million random points
```

```

long long inside_circle = 0;

int threads = omp_get_max_threads();
printf("Reg No: 240968040, Name: Devarjya\n");
printf("Total Points: %lld | Threads: %d\n", total_points, threads);

double start_time = omp_get_wtime();

#pragma omp parallel reduction(+:inside_circle)
{

    unsigned int seed = (unsigned int)(omp_get_thread_num() + 1) * 12345;

    #pragma omp for
    for (long long i = 0; i < total_points; i++) {
        // Generate random x and y between 0.0 and 1.0 using re-entrant or thread-safe pseudo-
random math
        // rand_r requires a seed pointer
        int rand_val_x = rand_r(&seed);
        int rand_val_y = rand_r(&seed);

        double x = (double)rand_val_x / RAND_MAX;
        double y = (double)rand_val_y / RAND_MAX;

        // Check if point lies inside the quarter circle ( $x^2 + y^2 \leq 1$ )
        if ((x * x + y * y) <= 1.0) {
            inside_circle++;
        }
    }
}

double end_time = omp_get_wtime();
double pi_estimate = 4.0 * (double)inside_circle / (double)total_points;

printf("Estimated Pi : %.6f\n", pi_estimate);
printf("Time Taken : %.4f seconds\n", end_time - start_time);

return 0;
}

```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=1
pcap1@PC-36:~/Documents/240968040/week 3$ ./q6
Reg No: 240968040, Name: Devarjya
Total Points: 10000000 | Threads: 1
Estimated Pi : 3.141386
Time Taken : 0.1388 seconds
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=2
pcap1@PC-36:~/Documents/240968040/week 3$ ./q6
Reg No: 240968040, Name: Devarjya
Total Points: 10000000 | Threads: 2
Estimated Pi : 3.140670
Time Taken : 0.0663 seconds
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=4
pcap1@PC-36:~/Documents/240968040/week 3$ ./q6
Reg No: 240968040, Name: Devarjya
Total Points: 10000000 | Threads: 4
Estimated Pi : 3.140754
Time Taken : 0.0360 seconds
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=8
pcap1@PC-36:~/Documents/240968040/week 3$ ./q6
Reg No: 240968040, Name: Devarjya
Total Points: 10000000 | Threads: 8
Estimated Pi : 3.140711
Time Taken : 0.0198 seconds
pcap1@PC-36:~/Documents/240968040/week 3$ export OMP_NUM_THREADS=16
pcap1@PC-36:~/Documents/240968040/week 3$ ./q6
Reg No: 240968040, Name: Devarjya
Total Points: 10000000 | Threads: 16
Estimated Pi : 3.141718
Time Taken : 0.0140 seconds
```